

Linux capabilities

Las *capacidades* de Linux son unidades distintas en las que se dividen los privilegios tradicionalmente asociados con el superusuario (root). Estas se pueden habilitar y deshabilitar de forma independiente, y son un atributo por hilo.

A continuación, se presenta una lista de las capacidades implementadas en Linux y las operaciones o comportamientos que cada capacidad permite:

- **CAP_AUDIT_CONTROL** (desde Linux 2.6.11)
 - Habilitar y deshabilitar la auditoría del kernel.
 - Cambiar las reglas de filtro de auditoría.
 - Recuperar el estado de auditoría y las reglas de filtrado.
- **CAP_AUDIT_READ** (desde Linux 3.16)
 - Permitir la lectura del registro de auditoría a través de un socket netlink de multidifusión.
- **CAP_AUDIT_WRITE** (desde Linux 2.6.11)
 - Escribir registros en el log de auditoría del kernel.
- **CAP_BLOCK_SUSPEND** (desde Linux 3.5)
 - Emplear características que pueden bloquear la suspensión del sistema (epoll(7) **EPOLLWAKEUP**, */proc/sys/wake_lock*).
- **CAP_BPF** (desde Linux 5.8)
 - Emplear operaciones BPF privilegiadas. Esta capacidad se añadió en Linux 5.8 para separar la funcionalidad BPF de la sobrecargada **CAP_SYS_ADMIN**.
- **CAP_CHECKPOINT_RESTORE** (desde Linux 5.9)
 - Actualizar */proc/sys/kernel/ns_last_pid*.
 - Emplear la característica *set_tid* de clone3(2).
 - Leer el contenido de los enlaces simbólicos en */proc/pid/map_files* para otros procesos. Esta capacidad se añadió en Linux 5.9 para separar la funcionalidad de checkpoint/restore de la sobrecargada **CAP_SYS_ADMIN**.
- **CAP_CHOWN**

- Realizar cambios arbitrarios en los UID y GID de los archivos.
- **CAP_DAC_OVERRIDE**
 - Omitir las verificaciones de permisos de lectura, escritura y ejecución de archivos.
- **CAP_DAC_READ_SEARCH**
 - Omitir las verificaciones de permisos de lectura de archivos y las verificaciones de permisos de lectura y ejecución de directorios.
 - Invocar `open_by_handle_at(2)`.
 - Usar el flag **AT_EMPTY_PATH** de `linkat(2)` para crear un enlace a un archivo referido por un descriptor de archivo.
- **CAP_FOWNER**
 - Omitir las verificaciones de permisos en operaciones que normalmente requieren que el UID del sistema de archivos del proceso coincida con el UID del archivo (por ejemplo, `chmod(2)`, `utime(2)`), excluyendo las operaciones cubiertas por **CAP_DAC_OVERRIDE** y **CAP_DAC_READ_SEARCH**.
 - Establecer flags de inodo (ver `FS_IOC_SETFLAGS(2const)`) en archivos arbitrarios.
 - Establecer Listas de Control de Acceso (ACLs) en archivos arbitrarios.
 - Ignorar el sticky bit del directorio en la eliminación de archivos.
 - Modificar atributos extendidos de *usuario* en directorios sticky propiedad de cualquier usuario.
 - Especificar **O_NOATIME** para archivos arbitrarios en `open(2)` y `fcntl(2)`.
- **CAP_FSETID**
 - No borrar los bits de modo set-user-ID y set-group-ID cuando se modifica un archivo.
 - Establecer el bit set-group-ID para un archivo cuyo GID no coincide con el sistema de archivos o con cualquiera de los GIDs suplementarios del proceso que realiza la llamada.
- **CAP_IPC_LOCK**
 - Bloquear memoria (`mlock(2)`, `mlockall(2)`, `mmap(2)`, `shmctl(2)`).
 - Asignar memoria usando huge pages (`memfd_create(2)`, `mmap(2)`, `shmctl(2)`).
- **CAP_IPC_OWNER**

- Omitir las verificaciones de permisos para operaciones en objetos IPC de System V.
- **CAP_KILL**
 - Omitir las verificaciones de permisos para enviar señales (ver kill(2)). Esto incluye el uso de la operación ioctl(2) **KDSIGACCEPT**.
- **CAP_LEASE** (desde Linux 2.4)
 - Establecer leases en archivos arbitrarios (ver fcntl(2)).
- **CAP_LINUX_IMMUTABLE**
 - Establecer los flags de inodo **FS_APPEND_FL** y **FS_IMMUTABLE_FL** (ver FS_IOC_SETFLAGS(2const)).
- **CAP_MAC_ADMIN** (desde Linux 2.6.25)
 - Permitir la configuración MAC o cambios de estado. Implementado para el Módulo de Seguridad de Linux (LSM) Smack.
- **CAP_MAC_OVERRIDE** (desde Linux 2.6.25)
 - Anular el Control de Acceso Obligatorio (MAC). Implementado para el LSM Smack.
- **CAP_MKNOD** (desde Linux 2.4)
 - Crear archivos especiales usando mknod(2).
- **CAP_NET_ADMIN**
 - Realizar varias operaciones relacionadas con la red, incluyendo:
 - Configuración de interfaz.
 - Administración de firewall IP, masquerading y contabilidad.
 - Modificar tablas de enrutamiento.
 - Vincular a cualquier dirección para proxy transparente.
 - Establecer tipo de servicio (TOS).
 - Borrar estadísticas del controlador.
 - Establecer modo promiscuo.
 - Habilitar multidifusión.
 - Usar setsockopt(2) para establecer las siguientes opciones de socket: **SO_DEBUG**, **SO_MARK**, **SO_PRIORITY** (para una prioridad fuera del rango 0 a 6), **SO_RCVBUFFORCE** y **SO_SNDBUFFORCE**.
- **CAP_NET_BIND_SERVICE**

- Vincular un socket a puertos privilegiados de dominio de Internet (números de puerto menores de 1024).
- **CAP_NET_BROADCAST**
 - (No utilizada) Realizar transmisiones de socket y escuchar multidifusiones.
- **CAP_NET_RAW**
 - Usar sockets RAW y PACKET.
 - Vincular a cualquier dirección para proxy transparente.
- **CAP_PERFMON** (desde Linux 5.8)
 - Emplear varios mecanismos de monitoreo de rendimiento, incluyendo:
 - Llamar a `perf_event_open(2)`.
 - Emplear varias operaciones BPF que tienen implicaciones de rendimiento. Esta capacidad se añadió en Linux 5.8 para separar la funcionalidad de monitoreo de rendimiento de la sobrecargada **CAP_SYS_ADMIN**.
- **CAP_SETGID**
 - Realizar manipulaciones arbitrarias de los GID del proceso y la lista de GID suplementarios.
 - Forjar GID al pasar credenciales de socket a través de sockets de dominio UNIX.
 - Escribir un mapeo de ID de grupo en un espacio de nombres de usuario.
- **CAP_SETPCAP** (desde Linux 2.6.24)
 - Establecer capacidades arbitrarias en un archivo. Desde Linux 5.12, esta capacidad también es necesaria para mapear el ID de usuario 0 en un nuevo espacio de nombres de usuario.
- **CAP_SETPCAP**
 - Si se admiten las capacidades de archivo (desde Linux 2.6.24): añadir cualquier capacidad del conjunto de límites del hilo que llama a su conjunto heredable; eliminar capacidades del conjunto de límites (a través de `prctl(2)` **PR_CAPBSET_DROP**); realizar cambios en los flags *securebits*.

- Si no se admiten las capacidades de archivo (antes de Linux 2.6.24): otorgar o eliminar cualquier capacidad en el conjunto de capacidades permitidas del llamador a o de cualquier otro proceso.

- **CAP_SETUID**

- Realizar manipulaciones arbitrarias de los UID del proceso (setuid(2), setreuid(2), setresuid(2), setfsuid(2)).
- Forjar UID al pasar credenciales de socket a través de sockets de dominio UNIX.
- Escribir un mapeo de ID de usuario en un espacio de nombres de usuario.

- **CAP_SYS_ADMIN**

- Realizar una serie de operaciones de administración del sistema, incluyendo: quotactl(2), mount(2), umount(2), pivot_root(2), swapon(2), swapoff(2), sethostname(2) y setdomainname(2).
- Realizar operaciones privilegiadas de syslog(2) (desde Linux 2.6.37, se debe usar **CAP_SYSLOG** para permitir tales operaciones).
- Realizar el comando **VM86_REQUEST_IRQ** vm86(2).
- Acceder a la misma funcionalidad de checkpoint/restore que se rige por **CAP_CHECKPOINT_RESTORE** (pero esta última capacidad más débil se prefiere para acceder a esa funcionalidad).
- Realizar las mismas operaciones BPF que se rigen por **CAP_BPF** (pero esta última capacidad más débil se prefiere para acceder a esa funcionalidad).
- Emplear los mismos mecanismos de monitoreo de rendimiento que se rigen por **CAP_PERFMON** (pero esta última capacidad más débil se prefiere para acceder a esa funcionalidad).
- Realizar operaciones **IPC_SET** e **IPC_RMID** en objetos IPC de System V arbitrarios.
- Anular el límite de recursos **RLIMIT_NPROC**.
- Realizar operaciones en atributos extendidos *trusted* y *security*.
- Usar lookup_dcookie(2).
- Usar ioprio_set(2) para asignar clases de programación de I/O **IOPRIO_CLASS_RT** y (antes de Linux 2.6.25) **IOPRIO_CLASS_IDLE**.

- Forjar PID al pasar credenciales de socket a través de sockets de dominio UNIX.
- Exceder `/proc/sys/fs/file-max`, el límite de todo el sistema en el número de archivos abiertos, en llamadas al sistema que abren archivos (por ejemplo, `accept(2)`, `execve(2)`, `open(2)`, `pipe(2)`).
- Emplear flags **CLONE_*** que crean nuevos espacios de nombres con `clone(2)` y `unshare(2)` (pero, desde Linux 3.8, crear espacios de nombres de usuario no requiere ninguna capacidad).
- Acceder a información privilegiada de eventos *perf*.
- Llamar a `setns(2)` (requiere **CAP_SYS_ADMIN** en el espacio de nombres *target*).
- Llamar a `fanotify_init(2)`.
- Realizar operaciones privilegiadas **KEYCTL_CHOWN** y **KEYCTL_SETPERM** `keyctl(2)`.
- Realizar la operación `madvise(2)` **MADV_HWPOISON**.
- Emplear el `ioctl(2)` **TIOCSTI** para insertar caracteres en la cola de entrada de un terminal que no sea el terminal de control del llamador.
- Emplear la llamada al sistema obsoleta `nfsservctl(2)`.
- Emplear la llamada al sistema obsoleta `bdflush(2)`.
- Realizar varias operaciones privilegiadas `ioctl(2)` de dispositivos de bloque.
- Realizar varias operaciones privilegiadas `ioctl(2)` de sistemas de archivos.
- Realizar operaciones privilegiadas `ioctl(2)` en el dispositivo `/dev/random`.
- Instalar un filtro `seccomp(2)` sin tener que establecer primero el atributo de hilo *no_new_privs*.
- Modificar las reglas de permitir/denegar para los grupos de control de dispositivos.
- Emplear la operación `ptrace(2)` **PTRACE_SECCOMP_GET_FILTER** para volcar los filtros `seccomp` del tracee.
- Emplear la operación `ptrace(2)` **PTRACE_SETOPTIONS** para suspender las protecciones `seccomp` del tracee (es decir, el flag **PTRACE_O_SUSPEND_SECCOMP**).

- Realizar operaciones administrativas en muchos controladores de dispositivos.
- Modificar valores nice de autogrupos escribiendo en */proc/pid/autogroup*.
- **CAP_SYS_BOOT**
 - Usar `reboot(2)` y `kexec_load(2)`.
- **CAP_SYS_CHROOT**
 - Usar `chroot(2)`.
 - Cambiar espacios de nombres de montaje usando `setns(2)`.
- **CAP_SYS_MODULE**
 - Cargar y descargar módulos del kernel (ver `init_module(2)` y `delete_module(2)`).
 - Antes de Linux 2.6.25: eliminar capacidades del conjunto de límites de capacidades de todo el sistema.
- **CAP_SYS_NICE**
 - Reducir el valor nice del proceso (`nice(2)`, `setpriority(2)`) y cambiar el valor nice para procesos arbitrarios.
 - Establecer políticas de programación en tiempo real para el proceso que llama, y establecer políticas y prioridades de programación para procesos arbitrarios (`sched_setscheduler(2)`, `sched_setparam(2)`, `sched_setattr(2)`).
 - Establecer afinidad de CPU para procesos arbitrarios (`sched_setaffinity(2)`).
 - Establecer clase y prioridad de programación de I/O para procesos arbitrarios (`ioprio_set(2)`).
 - Aplicar `migrate_pages(2)` a procesos arbitrarios y permitir que los procesos sean migrados a nodos arbitrarios.
 - Aplicar `move_pages(2)` a procesos arbitrarios.
 - Usar el flag **MPOL_MF_MOVE_ALL** con `mbind(2)` y `move_pages(2)`.
- **CAP_SYS_PACCT**
 - Usar `acct(2)`.
- **CAP_SYS_PTRACE**
 - Rastrear procesos arbitrarios usando `ptrace(2)`.
 - Aplicar `get_robust_list(2)` a procesos arbitrarios.

- Transferir datos hacia o desde la memoria de procesos arbitrarios usando `process_vm_readv(2)` y `process_vm_writev(2)`.
- Inspeccionar procesos usando `kcmp(2)`.
- **CAP_SYS_RAWIO**
 - Realizar operaciones de puerto de E/S (`iopl(2)` e `ioperm(2)`).
 - Acceder a `/proc/kcore`.
 - Emplear la operación `ioctl(2)` **FIBMAP**.
 - Abrir dispositivos para acceder a registros específicos del modelo x86 (MSRs).
 - Actualizar `/proc/sys/vm/mmap_min_addr`.
 - Crear mapeos de memoria en direcciones por debajo del valor especificado por `/proc/sys/vm/mmap_min_addr`.
 - Mapear archivos en `/proc/bus/pci`.
 - Abrir `/dev/mem` y `/dev/kmem`.
 - Realizar varios comandos de dispositivos SCSI.
 - Realizar ciertas operaciones en dispositivos `hpsa(4)` y `cciss(4)`.
 - Realizar una serie de operaciones específicas del dispositivo en otros dispositivos.
- **CAP_SYS_RESOURCE**
 - Usar espacio reservado en sistemas de archivos `ext2`.
 - Realizar llamadas `ioctl(2)` controlando el journaling de `ext3`.
 - Anular límites de cuota de disco.
 - Aumentar límites de recursos (ver `setrlimit(2)`).
 - Anular límite de recursos **RLIMIT_NPROC**.
 - Anular el número máximo de consolas en la asignación de consola.
 - Anular el número máximo de mapas de teclado.
 - Permitir más de 64hz de interrupciones del reloj de tiempo real.
 - Aumentar el límite `msg_qbytes` para una cola de mensajes de System V por encima del límite en `/proc/sys/kernel/msgmnb`.
 - Permitir que el límite de recursos **RLIMIT_NOFILE** en el número de descriptors de archivo "en vuelo" se omita al pasar descriptors de archivo a otro proceso a través de un socket de dominio UNIX.
 - Anular el límite `/proc/sys/fs/pipe-size-max` al establecer la capacidad de una tubería usando el comando **F_SETPIPE_SZ** `fcntl(2)`.

- Usar **F_SETPIPE_SZ** para aumentar la capacidad de una tubería por encima del límite especificado por */proc/sys/fs/pipe-max-size*.
- Anular los límites */proc/sys/fs/mqueue/queues_max*, */proc/sys/fs/mqueue/msg_max* y */proc/sys/fs/mqueue/msgsize_max* al crear colas de mensajes POSIX.
- Emplear la operación `prctl(2)` **PR_SET_MM**.
- Establecer */proc/pid/oom_score_adj* a un valor inferior al valor establecido por última vez por un proceso con **CAP_SYS_RESOURCE**.
- **CAP_SYS_TIME**
 - Establecer reloj del sistema (`settimeofday(2)`, `stime(2)`, `adjtimex(2)`).
 - Establecer reloj de tiempo real (hardware).
- **CAP_SYS_TTY_CONFIG**
 - Usar `vhangup(2)`.
 - Emplear varias operaciones `ioctl(2)` privilegiadas en terminales virtuales.
- **CAP_SYSLOG** (desde Linux 2.6.37)
 - Realizar operaciones `syslog(2)` privilegiadas.
 - Ver direcciones del kernel expuestas a través de */proc* y otras interfaces cuando */proc/sys/kernel/kptr_restrict* tiene el valor 1.
- **CAP_WAKE_ALARM** (desde Linux 3.0)
 - Activar algo que despertará el sistema (establecer temporizadores **CLOCK_REALTIME_ALARM** y **CLOCK_BOOTTIME_ALARM**).